TRƯỜNG ĐẠI HỌC CỬU LONG • KHOA CNTT – TRÍ TUỆ NHÂN TẠO

BÀI GIẢNG KIỂM THỬ PHẦN MỀM

CHƯƠNG 2

KIỂM THỬ TRONG VÒNG ĐỜI PHÁT TRIỂN PHẦN MỀM

SDLC – Mô hình phát triển – Cấp độ kiểm thử – Loại kiểm thử – Kiểm thử bảo trì

Thời lượng: **5 tiết** (1 tiết = 45 phút)

Biên soạn: ThS. Lôi Thị Tú Trân

Vĩnh Long – 2026

Mục tiêu chương học

Kiến thức

Giải thích được vai trò của kiểm thử trong từng giai đoạn của vòng đời phát triển phần mềm (SDLC).

Kiến thức

So sánh được cách tổ chức kiểm thử trong Waterfall, V-Model, Iterative, Agile, Spiral và DevOps.

Kiến thức

Phân biệt được 4 cấp độ kiểm thử và các loại kiểm thử theo ISTQB.

Kỹ năng

Lựa chọn được mô hình phát triển và chiến lược kiểm thử phù hợp với đặc điểm dự án.

Kỹ năng

Thực hiện được phân tích tác động (impact analysis) và lập kế hoạch kiểm thử bảo trì.

Thái độ

Nhận thức tầm quan trọng của việc kiểm thử sớm và liên tục (shift-left, continuous testing).

Chuẩn tham chiếu: **ISTQB CTFL v4.0** – Chapter 2: Testing Throughout the SDLC.

Nội dung chương 2

- **2.1** Kiểm thử trong vòng đời phát triển phần mềm (SDLC)
- **2.2** Các mô hình phát triển: Waterfall – V-Model – Iterative – Agile – Spiral – DevOps
- **2.3** Kiểm thử "shift-left" và các cách tiếp cận hợp tác (TDD, ATDD, BDD)
- **2.4** Bốn cấp độ kiểm thử (Test Levels)
- **2.5** Kim tự tháp kiểm thử và Bốn góc phần tư kiểm thử
- **2.6** Các loại kiểm thử (Test Types)
- **2.7** Kiểm thử liên quan đến thay đổi: Confirmation & Regression
- **2.8** Kiểm thử trong bảo trì phần mềm (Maintenance Testing)
- **2.9** Bài tập vận dụng + Bài tập tự luyện + Câu hỏi ôn tập

THỜI LƯỢNG: ≈ 0,5 TIẾT

2.1

Kiểm thử trong vòng đời phát triển phần mềm

- Kiểm thử ở từng giai đoạn SDLC
- Nguyên tắc "kiểm thử tốt"

Hoạt động kiểm thử trong từng giai đoạn của SDLC

Giai đoạn SDLC	Hoạt động kiểm thử tương ứng	Sản phẩm
Phân tích yêu cầu	Rà soát (review) tài liệu yêu cầu: phát hiện mơ hồ, mâu thuẫn, thiếu sót, yêu cầu không kiểm thử được.	Test plan sơ bộ
Thiết kế	Xem xét tài liệu thiết kế, kiến trúc, mô hình dữ liệu; thiết kế test case mức hệ thống.	Test design
Lập trình	Lập trình viên viết **unit test**; phân tích tĩnh mã nguồn; review mã (code review).	Unit test, báo cáo static analysis
Kiểm thử	Thực hiện kiểm thử tích hợp, hệ thống, chấp nhận; ghi nhận và theo dõi lỗi.	Test log, defect report
Triển khai	Kiểm thử môi trường triển khai, **smoke test** sau khi cài đặt, kiểm thử cấu hình.	Biên bản nghiệm thu
Bảo trì	Kiểm thử xác nhận (confirmation) và **kiểm thử hồi quy (regression)** sau mỗi thay đổi.	Báo cáo hồi quy

Kết luận: kiểm thử không phải một giai đoạn nằm cuối, mà là hoạt động **xuyên suốt toàn bộ vòng đời**.

Đặc điểm của kiểm thử "tốt" trong mọi mô hình SDLC (ISTQB)

Mỗi hoạt động phát triển có hoạt động kiểm thử tương ứng

Không có giai đoạn phát triển nào mà không đi kèm hoạt động kiểm chứng.

Mỗi cấp độ kiểm thử có mục tiêu riêng

Mục tiêu đặc thù giúp tránh trùng lặp và bỏ sót giữa các cấp độ.

Phân tích & thiết kế test bắt đầu từ sớm

Ngay khi cơ sở kiểm thử (yêu cầu, thiết kế) sẵn sàng – nguyên tắc shift-left.

Tester tham gia review ngay từ bản nháp đầu tiên

Phát hiện khiếm khuyết ở nơi rẻ nhất để sửa: trong tài liệu.

Mô hình SDLC ảnh hưởng đến: **thời điểm, phạm vi, mức độ tự động hoá và mức độ chính thức** của hoạt động kiểm thử.

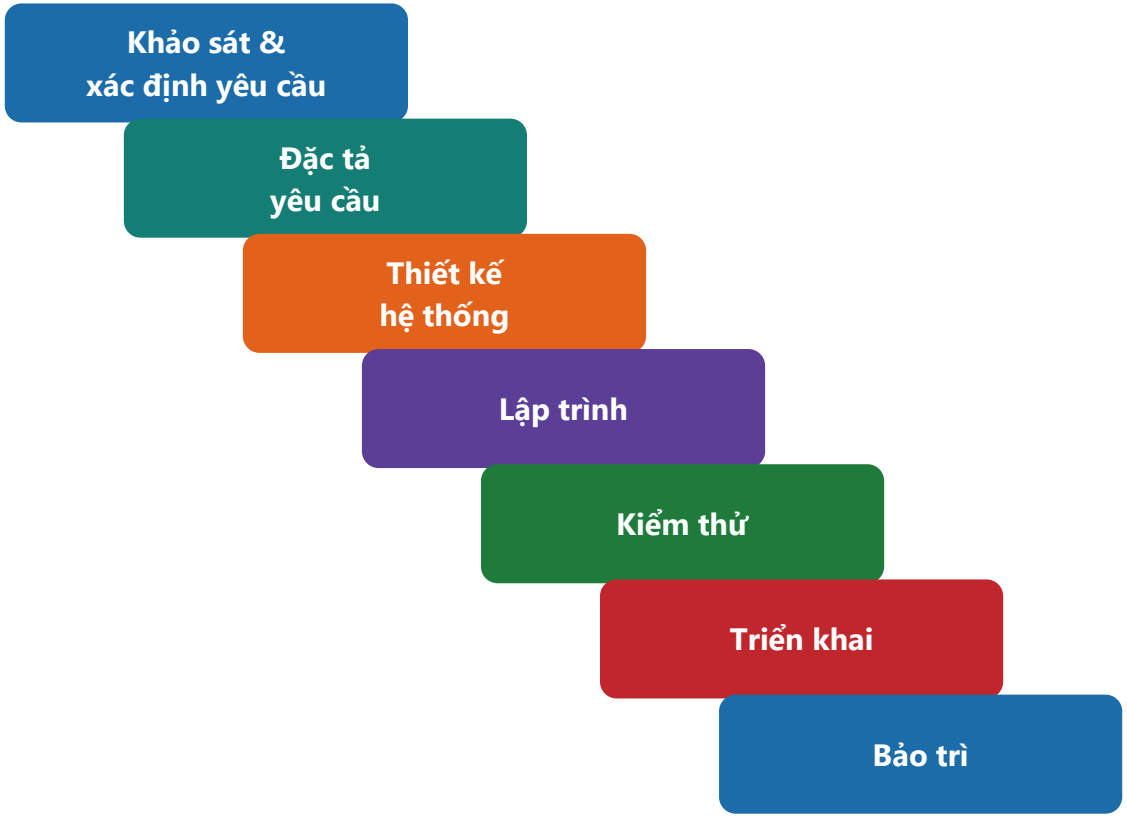
THỜI LƯỢNG: ≈ 1 TIẾT

2.2

Các mô hình vòng đời phát triển phần mềm

- Waterfall – V-Model – Iterative
- Agile – Spiral – DevOps

Mô hình thác nước (Waterfall Model)



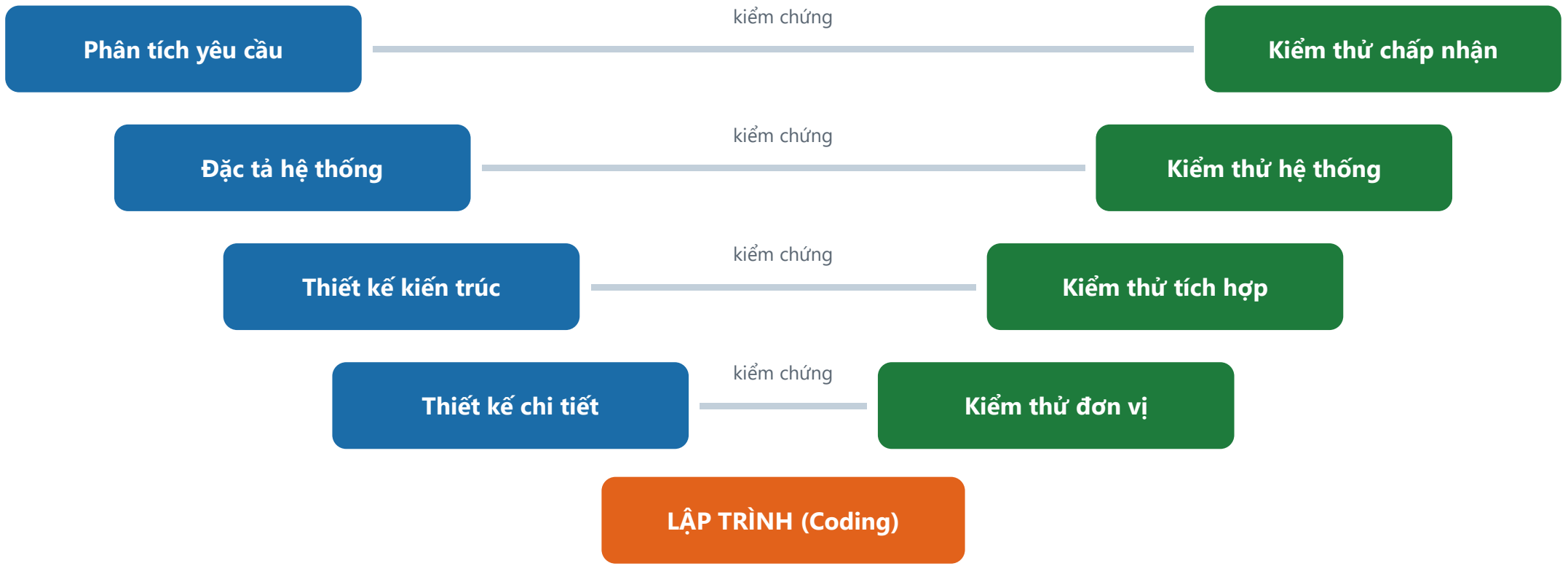
ĐẶC ĐIỂM

- Các bước thực hiện **tuần tự**, khó quay lại
- Kiểm thử diễn ra **sau khi lập trình xong**
- Phù hợp dự án yêu cầu rõ ràng, ít thay đổi
- Tài liệu đầy đủ, dễ quản lý hợp đồng

NHƯỢC ĐIỂM

- **Phát hiện lỗi muộn** ⇒ chi phí sửa rất cao
- Khó thích nghi khi yêu cầu thay đổi
- Khách hàng chỉ thấy sản phẩm ở giai đoạn cuối
- Kiểm thử thường bị "bóp" thời gian nếu dự án trễ

Mô hình chữ V (V-Model)



Mỗi giai đoạn phát triển (nhánh trái) có **một cấp độ kiểm thử tương ứng** (nhánh phải). Test được **thiết kế song song** với hoạt động phát triển ⇒ phát hiện lỗi sớm hơn Waterfall.

Mô hình lặp (Iterative) và Mô hình gia tăng (Incremental)

MÔ HÌNH LẶP – Iterative

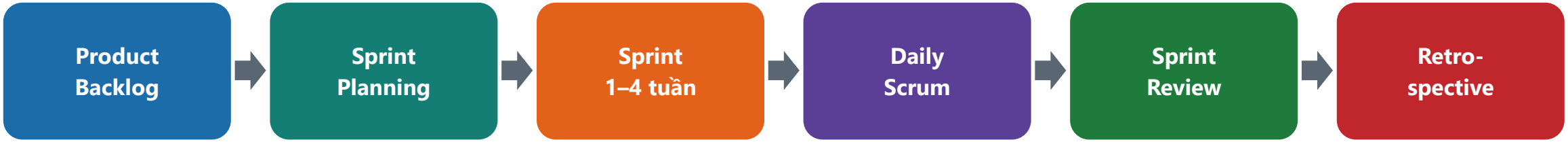
- Sản phẩm được **làm đi làm lại**, mỗi vòng lặp hoàn thiện hơn vòng trước
- Mỗi vòng gồm đủ: phân tích → thiết kế → lập trình → **kiểm thử**
- Ưu điểm: phát hiện lỗi sớm, dễ điều chỉnh yêu cầu
- Thách thức kiểm thử: **hồi quy tăng dần** qua từng vòng lặp

MÔ HÌNH GIA TĂNG – Incremental

- Hệ thống được **chia nhỏ thành các phần chức năng** và bàn giao dần
- Mỗi phần được kiểm thử độc lập rồi **tích hợp** vào hệ thống chung
- Ưu điểm: khách hàng dùng được sản phẩm sớm
- Thách thức kiểm thử: nhiều đợt **kiểm thử tích hợp**, cần bộ hồi quy tự động

Cả hai mô hình đều đòi hỏi **kiểm thử hồi quy tự động** vì số lần chạy lại test tăng nhanh theo số vòng lặp.

Mô hình Agile – Scrum



KIỂM THỬ DIỄN RA LIÊN TỤC TRONG SUỐT SPRINT – tester tham gia từ ngày đầu tiên

Tester tham gia sớm

Cùng viết **tiêu chí chấp nhận (acceptance criteria)** trong buổi refinement, trước khi code.

Tự động hoá là bắt buộc

Sprint ngắn ⇒ phải có **bộ hồi quy tự động** chạy trong CI, nếu không sẽ không kịp.

Chất lượng là việc của cả nhóm

Whole-team approach: dev, tester, PO cùng chịu trách nhiệm về chất lượng.

Mô hình xoắn ốc (Spiral Model) – mỗi vòng lặp gồm 4 hoạt động

BƯỚC 1

Lập kế hoạch

Xác định mục tiêu, phạm vi, ràng buộc và phương án thay thế của vòng lặp.

BƯỚC 2

Phân tích & đánh giá rủi ro

Nhận diện rủi ro kỹ thuật/nghiệp vụ, làm bản mẫu (prototype) để giảm rủi ro.

BƯỚC 3

Phát triển

Thiết kế và lập trình phần chức năng đã chọn cho vòng lặp này.

BƯỚC 4

Kiểm thử & đánh giá

Kiểm thử phần vừa xây dựng, khách hàng đánh giá và quyết định vòng tiếp theo.

Điểm nổi bật: **quản lý rủi ro là trung tâm**. Phù hợp dự án **lớn, phức tạp, yêu cầu chưa rõ ràng** (hệ thống quốc phòng, tài chính lớn). Nhược điểm: tốn kém, cần chuyên gia phân tích rủi ro.

DevOps và Kiểm thử liên tục (Continuous Testing)



Lợi ích: phản hồi nhanh, phát hiện lỗi tức thì. **Thách thức:** phải đầu tư hạ tầng CI/CD và **bộ kiểm thử tự động ổn định** (ít test "flaky").

So sánh cách tổ chức kiểm thử trong các mô hình SDLC

Mô hình	Thời điểm kiểm thử	Ưu điểm chính	Phù hợp với
Waterfall	Sau khi lập trình xong	Tài liệu đầy đủ, dễ quản lý	Yêu cầu rõ, ít thay đổi
V-Model	Thiết kế test song song từng giai đoạn	Phát hiện lỗi sớm, truy vết tốt	Hệ thống an toàn cao (y tế, hàng không)
Iterative	Kiểm thử ở mỗi vòng lặp	Điều chỉnh linh hoạt	Dự án vừa, yêu cầu tiến hoá
Agile / Scrum	Liên tục trong mỗi sprint	Phản hồi nhanh, gần khách hàng	Yêu cầu thay đổi thường xuyên
Spiral	Mỗi vòng, sau bước đánh giá rủi ro	Kiểm soát rủi ro tốt nhất	Dự án lớn, phức tạp, rủi ro cao
DevOps / CI-CD	Tự động, liên tục sau mỗi commit	Phát hành nhanh, chất lượng ổn định	Sản phẩm SaaS, phát hành thường xuyên

Shift-left và các cách tiếp cận hợp tác (Collaboration-based approaches)

Shift-left testing

Dịch chuyển hoạt động kiểm thử **về phía sớm** của vòng đời: review yêu cầu, phân tích tĩnh, unit test, kiểm thử API trước khi có UI.

TDD – Test-Driven Development

Viết **test trước**, sau đó viết mã vừa đủ để test pass, rồi tái cấu trúc (refactor). Chu trình **Red – Green – Refactor**.

ATDD – Acceptance TDD

Cả nhóm cùng viết **tiêu chí chấp nhận** dưới dạng test trước khi phát triển, dựa trên góc nhìn người dùng.

BDD – Behaviour-Driven Development

Mô tả hành vi mong đợi bằng ngôn ngữ tự nhiên **Given – When – Then**, để BA, dev, tester cùng hiểu.

Ví dụ BDD: *Given* sinh viên đã đăng ký 24 tín chỉ — *When* đăng ký thêm học phần 3 tín chỉ — *Then* hệ thống báo "Vượt giới hạn 25 tín chỉ" và không lưu đăng ký.

THỜI LƯỢNG: ≈ 1 TIẾT

2.4

Bốn cấp độ kiểm thử (Test Levels)

- Unit – Integration – System – Acceptance
- Mục tiêu, đối tượng và người thực hiện

Bốn cấp độ kiểm thử theo ISTQB

Cấp độ	Đối tượng kiểm thử	Cơ sở kiểm thử	Người thực hiện
1. Kiểm thử đơn vị (Unit / Component)	Hàm, phương thức, lớp, module nhỏ nhất	Thiết kế chi tiết, mã nguồn	Lập trình viên
2. Kiểm thử tích hợp (Integration)	Giao tiếp giữa các module, giữa các hệ thống, API	Thiết kế kiến trúc, đặc tả giao diện	Developer / Tester
3. Kiểm thử hệ thống (System)	Toàn bộ hệ thống hoàn chỉnh	Đặc tả yêu cầu (chức năng & phi chức năng)	Tester độc lập
4. Kiểm thử chấp nhận (Acceptance)	Hệ thống trong ngữ cảnh nghiệp vụ thực tế	Yêu cầu nghiệp vụ, quy trình, hợp đồng	Khách hàng / người dùng

Lưu ý: cấp độ kiểm thử phân biệt bởi **đối tượng, mục tiêu, cơ sở kiểm thử và người thực hiện** – không đơn thuần là thứ tự thời gian.

Kiểm thử tích hợp: hai loại và ba chiến lược

HAI LOẠI TÍCH HỢP

- **Tích hợp thành phần (Component integration)**
  - Kiểm tra tương tác giữa các module **bên trong** một hệ thống
  - Thường tự động hoá, chạy trong CI
- **Tích hợp hệ thống (System integration)**
  - Kiểm tra tương tác **giữa các hệ thống** hoặc với dịch vụ ngoài
  - Ví dụ: hệ thống bán hàng ⇄ cổng thanh toán VNPay

BA CHIẾN LƯỢC TÍCH HỢP

- **Big Bang:** ghép tất cả cùng lúc rồi test. Nhanh nhưng **rất khó khoanh vùng lỗi**
- **Top-down:** từ module cấp cao xuống, dùng **stub** thay module chưa có
- **Bottom-up:** từ module cấp thấp lên, dùng **driver** để gọi module
- **Stub** = module giả bị gọi; **Driver** = module giả gọi đến

Càng ghép nhiều thành phần cùng lúc, càng **khó xác định nguyên nhân lỗi** ⇒ nên tích hợp **từng bước nhỏ (incremental)**.

Các dạng kiểm thử chấp nhận (Acceptance Testing)

UAT – User Acceptance Testing

Người dùng cuối kiểm tra hệ thống có đáp ứng **nhu cầu nghiệp vụ thực tế** hay không.

OAT – Operational Acceptance Testing

Bộ phận vận hành kiểm tra: sao lưu/phục hồi, cài đặt, gỡ cài, giám sát, bảo mật.

Contractual / Regulatory Acceptance

Kiểm tra tuân thủ **điều khoản hợp đồng** và **quy định pháp luật, tiêu chuẩn ngành**.

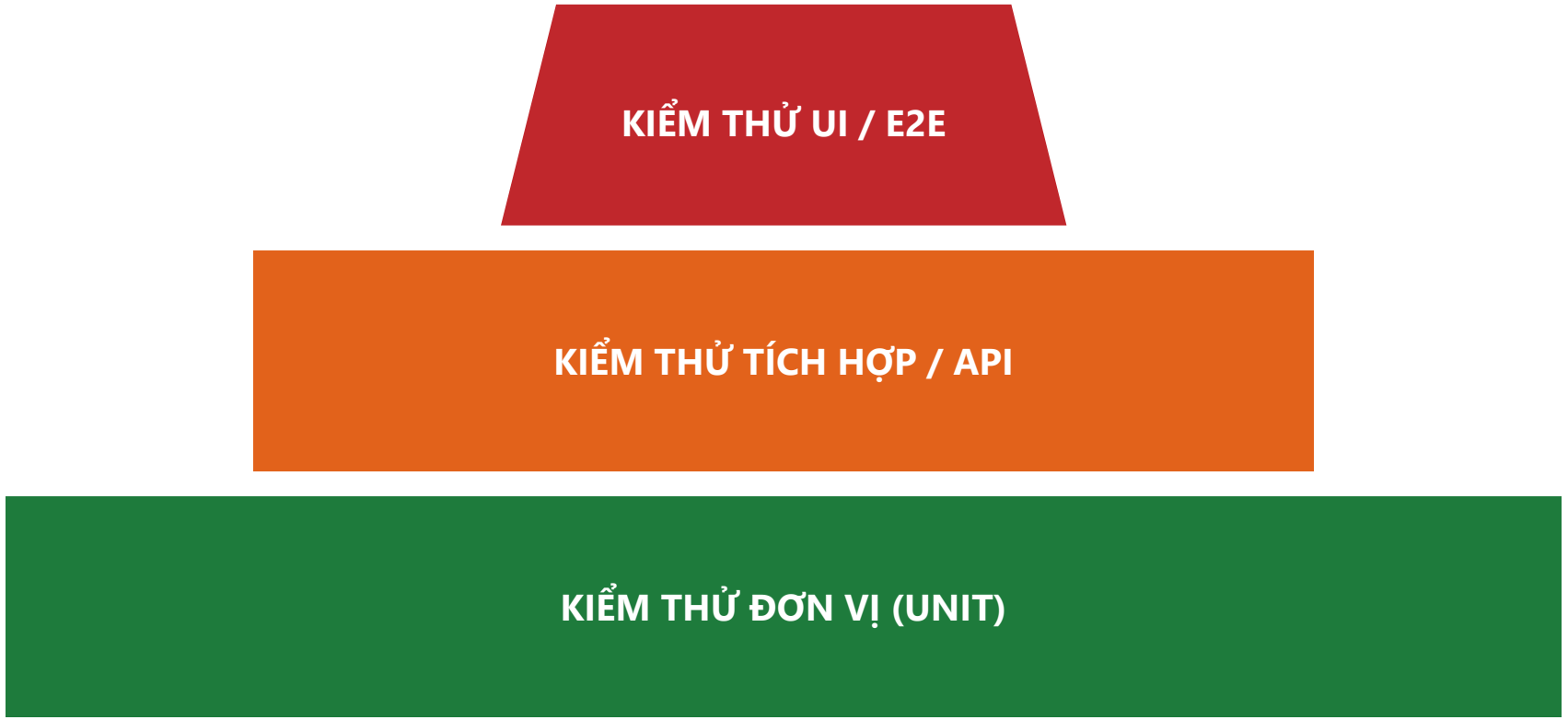
Alpha & Beta Testing

Alpha: người dùng thử tại nơi phát triển.

Beta: người dùng thật thử tại môi trường của họ.

Kết quả kiểm thử chấp nhận là căn cứ cho quyết định **nghiệm thu và đưa hệ thống vào vận hành**.

Kim tự tháp kiểm thử (Test Pyramid)



Ít • Chậm • Đắt • Dễ vỡ (flaky)

Vừa • Tốc độ trung bình • Ổn định hơn

Rất nhiều • Rất nhanh • Rẻ • Ổn định

Nguyên tắc: càng lên cao, test càng **chậm – đắt – khó bảo trì** ⇒ đặt phần lớn công sức tự động hoá ở **tầng đáy (unit)**. Kim tự tháp lộn ngược (nhiều test UI, ít unit test) là **dấu hiệu xấu (ice-cream cone anti-pattern)**.

THỜI LƯỢNG: ≈ 1 TIẾT

2.6

Các loại kiểm thử (Test Types)

- Chức năng – Phi chức năng
- Cấu trúc – Liên quan thay đổi

Bốn nhóm loại kiểm thử theo ISTQB

1. Kiểm thử chức năng (Functional Testing)

Kiểm tra hệ thống **làm gì**: các chức năng có đúng đặc tả không?

Ví dụ: đăng nhập, tìm kiếm, thanh toán, tính học phí.

Cơ sở: tài liệu yêu cầu, user story.

2. Kiểm thử phi chức năng (Non-functional Testing)

Kiểm tra hệ thống **hoạt động tốt như thế nào**: hiệu năng, bảo mật, khả năng sử dụng, độ tin cậy, tương thích.

Cơ sở: ISO/IEC 25010.

3. Kiểm thử cấu trúc (White-box Testing)

Dựa trên **cấu trúc bên trong**: câu lệnh, nhánh, đường đi, luồng dữ liệu.

Đo **độ bao phủ (coverage)**.

Thường do lập trình viên thực hiện.

4. Kiểm thử liên quan thay đổi (Change-related Testing)

Thực hiện khi hệ thống được sửa đổi:

Confirmation testing – xác nhận lỗi đã sửa.

Regression testing – đảm bảo chức năng cũ không hỏng.

Loại kiểm thử ⊥ Cấp độ kiểm thử: một loại có thể áp dụng ở **nhiều cấp độ**. Ví dụ kiểm thử hiệu năng có thể làm ở mức unit, API và hệ thống.

Các loại kiểm thử phi chức năng thường gặp

Kiểm thử hiệu năng (Performance)

Load test, stress test, spike test, endurance test. Đo thời gian phản hồi, thông lượng.

Kiểm thử bảo mật (Security)

Phân quyền, SQL injection, XSS, mã hoá dữ liệu, quản lý phiên đăng nhập.

Kiểm thử khả năng sử dụng (Usability)

Số thao tác để hoàn thành tác vụ, tính rõ ràng của thông báo lỗi, hỗ trợ người khuyết tật.

Kiểm thử tương thích (Compatibility)

Nhiều trình duyệt, hệ điều hành, kích thước màn hình, phiên bản thiết bị.

Kiểm thử độ tin cậy (Reliability)

Khả năng chịu lỗi, phục hồi sau sự cố (recovery), thời gian hoạt động liên tục.

Kiểm thử khả chuyển & bảo trì (Portability / Maintainability)

Cài đặt – gỡ cài đặt, nâng cấp phiên bản, độ phức tạp mã nguồn.

Kiểm thử liên quan đến thay đổi (Change-related Testing)

CONFIRMATION TESTING (Re-testing)

- **Mục đích:** xác nhận khiếm khuyết đã được sửa **đúng và triệt để**
- **Cách làm:** chạy lại **chính** test case đã phát hiện lỗi, với **cùng dữ liệu và cùng môi trường**
- **Phạm vi:** hẹp – chỉ tập trung vào lỗi đã sửa
- **Khi nào:** ngay sau khi developer báo đã fix
- Nếu vẫn fail ⇒ mở lại (reopen) defect

REGRESSION TESTING (Kiểm thử hồi quy)

- **Mục đích:** đảm bảo thay đổi **không phá vỡ** chức năng đang chạy tốt
- **Cách làm:** chạy lại bộ test case của các chức năng **liên quan** (hoặc toàn bộ)
- **Phạm vi:** rộng – xác định qua **phân tích tác động**
- **Khi nào:** sau mỗi bản build / phát hành
- Là **ứng viên số 1 cho kiểm thử tự động** vì lặp lại rất nhiều lần

Smoke test: bộ test rất ngắn chạy ngay sau khi build để xác nhận hệ thống "đủ sống" để kiểm thử sâu. **Sanity test:** kiểm tra nhanh một vùng chức năng vừa được sửa.

THỜI LƯỢNG: ≈ 0,5 TIẾT

2.8

Kiểm thử trong bảo trì phần mềm (Maintenance Testing)

- Nguyên nhân bảo trì
- Phân tích tác động
- Phạm vi kiểm thử bảo trì

Bốn nhóm nguyên nhân kích hoạt bảo trì (Maintenance Triggers)

Sửa lỗi – Corrective

Khắc phục defect phát hiện khi hệ thống đang vận hành. Có thể khẩn cấp (hot-fix).

Cải tiến – Perfective

Bổ sung chức năng mới, tối ưu hiệu năng, cải thiện giao diện theo phản hồi người dùng.

Thích nghi – Adaptive

Thay đổi để phù hợp môi trường mới: nâng cấp hệ điều hành, đổi trình duyệt, chuyển sang cloud.

Phòng ngừa – Preventive

Tái cấu trúc mã (refactoring), trả nợ kỹ thuật, nâng cấp thư viện để giảm rủi ro tương lai.

Ngoài ra còn có bảo trì do **di trú dữ liệu (migration)** và **ngừng vận hành hệ thống cũ (retirement)** – cần kiểm thử di trú và kiểm thử lưu trữ dữ liệu.

Quy trình kiểm thử bảo trì

BƯỚC 1

Tiếp nhận yêu cầu thay đổi

Ghi nhận change request / bug report từ vận hành, khách hàng.

BƯỚC 2

Phân tích tác động (Impact Analysis)

Xác định phần nào của hệ thống bị ảnh hưởng; dựa vào **ma trận truy vết** và kiến trúc.

BƯỚC 3

Xác định phạm vi kiểm thử

Chọn test case cần chạy lại: retest lỗi + regression vùng ảnh hưởng.

BƯỚC 4

Thực thi & đánh giá

Chạy confirmation + regression, đánh giá rủi ro còn lại, quyết định phát hành.

Phân tích tác động khó khăn khi: đặc tả lỗi thời hoặc thiếu, không có ma trận truy vết, người viết hệ thống đã nghỉ việc, tài liệu không được cập nhật.

THỜI LƯỢNG: ≈ 1 TIẾT

2.9

Bài tập vận dụng và câu hỏi ôn tập

- 4 bài tập
- 10 bài tập tự luyện
- 5 câu hỏi ôn tập

Lựa chọn mô hình phát triển phần mềm

- Một công ty phần mềm chuẩn bị phát triển các dự án sau:
  - **A.** Hệ thống quản lý thư viện cho một trường đại học – yêu cầu ổn định, ít thay đổi.
  - **B.** Ứng dụng đặt đồ ăn trực tuyến – thường xuyên cập nhật theo phản hồi khách hàng.
  - **C.** Hệ thống điều khiển thiết bị y tế – yêu cầu độ an toàn và tin cậy rất cao.
- **Yêu cầu:**
  - a) Chọn mô hình phát triển phù hợp cho từng dự án và giải thích lý do.
  - b) Phân tích cách tổ chức hoạt động kiểm thử trong từng mô hình đã chọn.

Xác định cấp độ kiểm thử

- Hệ thống **bán hàng trực tuyến** có các chức năng: đăng nhập, tìm kiếm sản phẩm, thêm vào giỏ hàng, thanh toán và quản lý đơn hàng.
- **Yêu cầu:**
  - a) Xác định các hoạt động cần thực hiện ở từng cấp độ: Unit – Integration – System – Acceptance.
  - b) Cho **ví dụ một test case cụ thể** tương ứng với mỗi cấp độ.
  - c) Cho biết ai là người thực hiện ở mỗi cấp độ.

Phân loại hoạt động kiểm thử

- Cho các tình huống sau, hãy **xác định loại kiểm thử** phù hợp và giải thích ngắn gọn:
  - **1.** Kiểm tra chức năng đăng nhập bằng tài khoản hợp lệ.
  - **2.** Đánh giá khả năng chịu tải của hệ thống khi có 5.000 người dùng đồng thời.
  - **3.** Kiểm tra lại lỗi đăng nhập sau khi lập trình viên đã sửa.
  - **4.** Kiểm tra các chức năng cũ sau khi bổ sung chức năng thanh toán trực tuyến.
  - **5.** Kiểm tra các nhánh điều kiện trong đoạn mã nguồn của chương trình.
- **Mở rộng:** với mỗi tình huống, cho biết nó thường được thực hiện ở **cấp độ kiểm thử** nào.

Kiểm thử trong bảo trì phần mềm

- Một **hệ thống quản lý sinh viên** đã được đưa vào sử dụng. Nhà phát triển thực hiện hai thay đổi:
  - ① Bổ sung chức năng **đăng ký học phần trực tuyến** (mới).
  - ② Sửa lỗi **tính học phí** (lỗi làm tròn số tiền miễn giảm).
- **Yêu cầu:**
  - a) Thực hiện **phân tích tác động (Impact Analysis)**.
  - b) Xác định các chức năng cần **Re-testing** và các chức năng cần **Regression Testing**.
  - c) Đề xuất kế hoạch kiểm thử đảm bảo hệ thống ổn định sau khi cập nhật.

Bộ 10 bài tập tự luyện (Chương 2)

- **Bài 1.** So sánh Waterfall và V-Model: điểm giống, điểm khác cơ bản về tổ chức kiểm thử.
- **Bài 2.** Vì sao trong Agile bắt buộc phải có kiểm thử tự động? Điều gì xảy ra nếu không có?
- **Bài 3.** Phân biệt **stub** và **driver**. Cho ví dụ khi tích hợp module Đăng nhập và module CSDL.
- **Bài 4.** Một nhóm có 20 unit test và 400 test UI tự động. Nhận xét về kim tự tháp kiểm thử của nhóm này và đề xuất cải thiện.
- **Bài 5.** Cho user story: *"Là sinh viên, tôi muốn hủy học phần đã đăng ký để điều chỉnh thời khoá biểu."* Hãy viết 3 kịch bản **BDD (Given–When–Then)**.
- **Bài 6.** Phân biệt **Smoke test**, **Sanity test** và **Regression test** về mục đích, phạm vi, thời điểm.
- **Bài 7.** Hệ thống ngân hàng nâng cấp từ Windows Server 2016 lên 2022. Đây là loại bảo trì nào? Cần kiểm thử những gì?
- **Bài 8.** Với ứng dụng đặt vé máy bay, hãy nêu 5 loại kiểm thử **phi chức năng** cần thực hiện và tiêu chí chấp nhận cho mỗi loại.
- **Bài 9.** Giải thích vì sao "cấp độ kiểm thử" và "loại kiểm thử" là hai khái niệm **độc lập**. Cho ví dụ một loại áp dụng ở 3 cấp độ.
- **Bài 10.** Dự án của bạn không có tài liệu đặc tả, người viết hệ thống đã nghỉ việc. Hãy nêu 4 khó khăn khi phân tích tác động và cách khắc phục.

Câu hỏi ôn tập chương 2

1. Trình bày vai trò của kiểm thử trong SDLC. Vì sao kiểm thử cần thực hiện **xuyên suốt** thay vì chỉ ở giai đoạn cuối?

2. So sánh Waterfall, V-Model, Iterative, Agile và Spiral về cách tổ chức kiểm thử. Mô hình nào phù hợp nhất với dự án **thường xuyên thay đổi yêu cầu**? Giải thích.

3. Trình bày **bốn cấp độ kiểm thử** theo ISTQB: mục tiêu, đối tượng, cơ sở kiểm thử và người thực hiện của mỗi cấp độ.

4. Phân biệt kiểm thử **chức năng – phi chức năng – cấu trúc – liên quan thay đổi**. Cho ví dụ minh hoạ mỗi loại.

5. Trình bày vai trò của kiểm thử trong bảo trì. Phân biệt **Re-testing** và **Regression Testing**; vì sao cần **phân tích tác động** trước khi kiểm thử?

6. Giải thích **kim tự tháp kiểm thử**. Vì sao nên có nhiều unit test hơn test giao diện?

Tóm tắt chương 2

- ✓ Kiểm thử là hoạt động **xuyên suốt SDLC**: mỗi giai đoạn phát triển đều có hoạt động kiểm chứng tương ứng.
- ✓ **Waterfall** kiểm thử ở cuối; **V-Model** gắn cấp độ kiểm thử với từng giai đoạn; **Agile/DevOps** kiểm thử liên tục và tự động.
- ✓ Bốn cấp độ kiểm thử – **Unit, Integration, System, Acceptance** – khác nhau về đối tượng, mục tiêu, cơ sở kiểm thử và người thực hiện.
- ✓ **Cấp độ** và **loại** kiểm thử là hai trục độc lập: một loại kiểm thử có thể áp dụng ở nhiều cấp độ khác nhau.
- ✓ **Confirmation testing** xác nhận lỗi đã sửa; **Regression testing** đảm bảo thay đổi không phá vỡ chức năng cũ – đây là ứng viên hàng đầu để tự động hoá.
- ✓ Kiểm thử bảo trì bắt đầu bằng **phân tích tác động** để khoanh vùng phạm vi, tránh vừa bỏ sót vừa lãng phí công sức.

Đánh giá kết quả học tập – Kiểm thử trong vòng đời phát triển phần mềm

BÀI QUIZ • 30%

- **15 câu trắc nghiệm**, thời gian 15 phút
- Làm vào **đầu buổi chiều** ngày 1, 2, 3
- Mỗi câu **0,67 điểm** – làm tròn thang 10
- Chấm chéo tại lớp, công bố đáp án ngay
- Lấy **trung bình 3 bài quiz**

BÀI KIỂM TRA VIẾT 45 PHÚT • 70%

- **20 câu trắc nghiệm + 1 bài tập vận dụng**
- Làm vào **đầu ngày 4**, phủ cả 6 chương
- Trắc nghiệm **5 điểm** – bài tập **5 điểm**
- Mỗi chương góp khoảng **3–4 câu** trắc nghiệm
- Bài tập vận dụng chọn **1 trong 6 đề** của các chương

CÁCH TÍNH ĐIỂM QUÁ TRÌNH

Điểm quá trình = **0,7 × điểm bài kiểm tra viết 45 phút** + **0,3 × trung bình 3 bài quiz**

Ví dụ: bài viết 8,0 và ba quiz lần lượt 7,0 – 8,0 – 9,0 (trung bình 8,0) → điểm quá trình = 0,7 × 8,0 + 0,3 × 8,0 = **8,0**

Quiz 15 câu – Chương 2 (phần 1/3)

Câu 1. Trong mô hình chữ V, hoạt động kiểm thử bắt đầu từ khi nào?
A. Khi code xong B. Khi bắt đầu lấy yêu cầu C. Sau unit test D. Trước khi bàn giao

Câu 2. Trong mô hình chữ V, "Thiết kế kiến trúc" tương ứng với cấp độ kiểm thử nào?
A. Unit B. Integration C. System D. Acceptance

Câu 3. Ai là người thực hiện kiểm thử đơn vị (unit test)?
A. Tester B. Developer C. Khách hàng D. Test lead

Câu 4. Lỗi "module A gửi ngày dạng dd/MM/yyyy, module B đọc theo MM/dd/yyyy" bị bắt ở cấp độ:
A. Unit B. Integration C. System D. Acceptance

Câu 5. Kiểm thử chấp nhận (acceptance test) chủ yếu do ai thực hiện?
A. Developer B. Tester C. Khách hàng / người dùng cuối D. Quản trị hệ thống

Quiz 15 câu – Chương 2 (phần 2/3)

Câu 6. Bộ phận vận hành kiểm tra khả năng sao lưu và khôi phục dữ liệu. Đây là:
A. UAT B. OAT C. Alpha test D. Beta test

Câu 7. Trong tích hợp top-down, STUB được dùng để thay thế:
A. Module gọi (cấp trên) B. Module được gọi (cấp dưới) C. Cơ sở dữ liệu D. Giao diện người dùng

Câu 8. Kim tự tháp kiểm thử khuyến nghị số lượng test NHIỀU NHẤT ở tầng:
A. Giao diện (UI/E2E) B. API C. Đơn vị (unit) D. Chấp nhận

Câu 9. Chạy lại đúng test case đã Fail sau khi dev sửa bug được gọi là:
A. Kiểm thử hồi quy B. Kiểm thử xác nhận C. Kiểm thử khói D. Kiểm thử chấp nhận

Câu 10. Chạy lại các test case KHÁC để bảo đảm việc sửa lỗi không làm hỏng chỗ đang chạy tốt:
A. Kiểm thử xác nhận B. Kiểm thử hồi quy C. Kiểm thử đơn vị D. Kiểm thử tải

Quiz 15 câu – Chương 2 (phần 3/3)

Câu 11. Kiểm thử áp lực (stress testing) nhằm mục đích:
A. Đo thời gian phản hồi ở tải bình thường B. Tìm ngưỡng gãy và cách hệ thống suy giảm C. Kiểm tra chức năng D. Kiểm tra bảo mật

Câu 12. Bộ Giáo dục đổi quy chế khiến hệ thống phải sửa. Đây là bảo trì:
A. Sửa lỗi B. Thích nghi C. Hoàn thiện D. Phòng ngừa

Câu 13. Hoạt động BẮT BUỘC trước khi kiểm thử bảo trì là:
A. Viết lại toàn bộ test case B. Phân tích tác động C. Tự động hoá toàn bộ D. Thay đổi mô hình SDLC

Câu 14. Trong DevOps, kiểm thử liên tục BẮT BUỘC phải có:
A. Tài liệu đặc tả 300 trang B. Kiểm thử tự động trong pipeline C. Nhóm kiểm thử độc lập bên ngoài D. Mô hình chữ V

Câu 15. Shift-left trong kiểm thử nghĩa là:
A. Chuyển việc test sang nhóm khác B. Đưa hoạt động kiểm thử về sớm hơn trong vòng đời C. Chỉ test phía bên trái giao diện D. Giảm số lượng test case

Ngân hàng câu trắc nghiệm cho bài kiểm tra 45 phút – Chương 2

Câu 1. Dự án phần mềm điều khiển thiết bị y tế, phải nộp hồ sơ kiểm định cho cơ quan quản lý với đầy đủ truy vết từ yêu cầu tới test case. Mô hình PHÙ HỢP NHẤT là:

A. Scrum B. DevOps C. Mô hình chữ V D. Kiểm thử thăm dò

Câu 2. Một dự án lặp bước sang đợt phát hành thứ năm. Khối lượng kiểm thử hồi quy:

A. Giảm dần vì hệ thống ổn định B. Không đổi C. Tăng dần vì phải test lại toàn bộ phần cũ D. Chỉ phụ thuộc số tester

Câu 3. Bộ hồi quy 500 test chạy tay mất 3 ngày, nhưng ngày mai phải phát hành. Cách xử lý HỢP LÝ NHẤT là:

A. Bỏ hoàn toàn hồi quy B. Chạy ngẫu nhiên 100 test C. Chọn theo phân tích tác động và báo cáo rủi ro phần chưa chạy D. Lùi phát hành vô thời hạn

Câu 4. Phát biểu nào về cấp độ và loại kiểm thử là ĐÚNG?

A. Mỗi cấp độ chỉ dùng được một loại B. Kiểm thử bảo mật chỉ chạy ở cấp hệ thống C. Một loại kiểm thử có thể chạy ở nhiều cấp độ D. Hồi quy là một cấp độ kiểm thử

Lập phương án kiểm thử theo vòng đời cho một dự án

Tình huống. Trường Đại học Cửu Long xây dựng **website đăng ký học phần trực tuyến**. Yêu cầu nghiệp vụ đã được Phòng Đào tạo ban hành rõ ràng và ít thay đổi. Hệ thống sẽ có khoảng **5.000 sinh viên truy cập đồng thời** trong ngày mở cổng đăng ký. Dự án kéo dài 4 tháng.

Yêu cầu.

a) Chọn **mô hình phát triển** phù hợp và giải thích dựa trên đặc điểm dự án. (1,0 đ)

b) Nêu **hoạt động kiểm thử** của tester trong từng giai đoạn, kể cả giai đoạn chưa có mã nguồn. (1,5 đ)

c) Với chức năng **"Đăng ký học phần"**, hãy cho **một ví dụ lỗi** có thể phát hiện ở mỗi cấp độ trong **bốn cấp độ** kiểm thử. (1,5 đ)

d) Nêu **hai loại kiểm thử phi chức năng** bắt buộc phải có cho hệ thống này và giải thích vì sao. (1,0 đ)